TrustShare: Intelligent Encrypted File Sharing and Digital Collaboration Management Platform

Project Report

Abstract

TrustShare is a secure file-sharing and digital collaboration platform that protects files with server-side AES-256-GCM encryption, controls how files are shared, and monitors user activity. The system uses a FastAPI backend, a Next.js frontend and a PostgreSQL database, with JWT-based authentication protecting every sensitive request. Users can upload files that are stored only in encrypted form, share them through revocable links, and review audit logs, analytics and security information from a dashboard. The project was built over four milestones covering core setup, encryption and sharing, monitoring and analytics, and testing. This report describes the design, implementation, testing results, limitations and future scope of the system.

1. Introduction

Organizations, schools and businesses exchange sensitive documents every day. Common sharing methods such as email attachments and public links give little control over who can open a file, for how long, or what happened to it afterwards. A file that leaks in storage or in transit is exposed in full.

TrustShare addresses this by encrypting every file before it is stored, requiring authentication for access, giving file owners control over sharing links, and recording activity in audit logs.

2. Objectives

- Build a secure platform where users can upload, store and share files.
- Protect file contents with AES-256-GCM encryption on the server.
- Provide authentication using JWT and protect file endpoints with access-control checks.

- Allow files to be shared through links that the owner can list and revoke.
- Record user and file activity in audit logs and present it in dashboards.
- Provide analytics and security pages, and notification support, for monitoring.

3. System Architecture

TrustShare follows a client-server architecture with a server-side encryption design.

1. The user interacts with the Next.js web application.
2. The frontend sends requests to the FastAPI backend, with a JWT in the request for protected operations.
3. The backend validates the token and permissions.
4. On upload, the backend encrypts the file with AES-256-GCM before storing it.
5. File metadata, users, sharing links and related records are stored in PostgreSQL.
6. On download, the backend checks authorization, decrypts the file in memory and returns it to the authorized user.
7. Activities such as uploads, downloads and sharing actions are written to audit logs.

[Insert the architecture diagram from the project brief here and add a note on which parts were implemented.]

4. Technology Stack

Layer	Technology
Frontend	Next.js, React, JavaScript, CSS
Backend	Python, FastAPI
Database	PostgreSQL
Authentication	JWT
Encryption	AES-256-GCM
Tools	Visual Studio Code, Git, GitHub, Swagger UI

5. Database Design

The PostgreSQL database stores the application data. The main entities are:

Entity	Purpose
Users	Account details, hashed credentials and role information
Files	File metadata such as name, size, owner and upload time, plus information needed to decrypt the stored file
Share links	Links created by owners to share files, with status so they can be revoked
Audit logs	Records of user and file activity for monitoring

[Insert an ER diagram or a table listing the actual columns from your models. Adjust the entity list above to match your code.]

6. Module Implementation

6.1 Authentication Module

Users can register and log in. On successful login the backend issues a JWT, which the frontend sends with protected API requests. A forgot-password workflow supports account recovery. The login page and dashboard were built in Next.js and connected to the backend.

[Screenshot: registration, login and dashboard]

6.2 File Management Module

Authenticated users can upload files, view a list of their files, open a file-detail page and download files. The file endpoints enforce authentication and access-control checks, so a user cannot reach files they are not authorized to access.

[Screenshot: upload, file list and file-detail page]

6.3 Encryption Module

Encryption is performed on the server using AES-256-GCM, an authenticated encryption mode. It provides confidentiality and also detects any tampering with the stored data.

Workflow:

1. The user uploads a file over an authenticated request.
2. The backend encrypts the file contents with AES-256-GCM.
3. Only the encrypted data is stored.
4. On download, the backend verifies the user's access, decrypts the file in memory and returns the original content.

[Screenshot or code excerpt of the encryption and decryption functions. Briefly describe how keys are managed in your implementation.]

6.4 Secure Sharing Module

File owners can create share links for their files, list the links they have created, and revoke a link so that it can no longer be used. Revocation gives the owner control over access after sharing.

[Screenshot: creating, listing and revoking share links]

6.5 Monitoring and Audit Logging Module

Relevant application activities are written to audit logs, and activity-monitoring functionality lets users review what happened to their files. Backend endpoints return this information to the dashboard pages.

[Screenshot: activity and audit log views]

6.6 Notifications, Analytics and Security Dashboards

The frontend includes dashboard, analytics and security pages that show file and activity information. Notification-related functionality and account management were also worked on, and an email-testing script was prepared to verify the SMTP configuration.

[Screenshot: analytics and security pages]

6.7 User Interface

In the final milestone the interface was improved with reusable components, including a TrustShare logo and a sidebar, and the dashboard, login, analytics, security and file-detail pages were updated for a consistent look.

7. Milestone Summary

Milestone	Work completed
1: Core setup and authentication	System design, FastAPI backend, registration and login, JWT authentication, Next.js frontend, file upload, listing and download APIs, forgot password, PostgreSQL
2: Encryption and secure sharing	AES-256-GCM encryption, encrypted storage, authenticated upload and download, share links, link listing and revocation, access-control checks
3: Monitoring, notifications and analytics	Audit logging, activity monitoring, dashboard, analytics and security pages, notification functionality, account management
4: Testing and documentation	API and workflow testing, interface improvements, frontend production build, email-testing script, documentation

8. Testing and Validation

Testing was carried out locally using the Swagger UI and the running application.

No.	Test	Expected result	Result
1	Backend health endpoint	Returns a successful status	Passed
2	Login and authenticated API request	Token issued and protected endpoint accessible	Passed
3	File upload	API returns uploaded file information	Passed
4	File listing	Uploaded files appear in the list	Passed

No.	Test	Expected result	Result
5	File download	File is returned correctly	Passed
6	Create share link	Link is created for the file	Passed
7	List share links	Created links are listed	Passed
8	Revoke share link	Link is revoked	Passed
9	Frontend production build	Build completes without errors	Passed

[Add screenshots of Swagger responses and the successful build. Add any further tests you ran, such as unauthorized access attempts.]

The testing so far verifies functional correctness of the main workflows. A full security assessment, including penetration testing and load testing, has not yet been carried out.

9. Deployment

The application currently runs locally.

Backend

```
cd backend
python -m uvicorn app.main:app --reload
```

The backend runs at http://localhost:8000 and the API documentation at http://localhost:8000/docs.

Frontend

```
cd frontend
npm install
npm run dev
```

The frontend runs at http://localhost:3000. A cloud deployment (for example the frontend on Vercel and the backend on a platform such as Render, with a hosted PostgreSQL database) is planned as future work and is not part of the current submission. [If you complete a deployment before submitting, replace this paragraph with the live URLs and screenshots.]

10. Challenges Faced

[Replace with your own experiences. Examples:]

- Connecting the Next.js frontend with the FastAPI backend, including token handling and cross-origin requests.
- Implementing encryption so that files are stored encrypted but can be returned correctly to authorized users.
- Designing share-link revocation and access checks so that revoked links stop working.
- Keeping the interface consistent across pages by moving shared elements into reusable components.

11. Limitations

The following items from the original project brief are not completed in the current version, or have not been verified:

- Cloud deployment and production infrastructure.
- Docker-based deployment and CI/CD pipeline.
- Full security testing and validation (penetration testing, vulnerability scanning, load testing).
- Multi-factor authentication and OAuth2 login. [Remove from this list if you implemented them.]
- MongoDB, Redis and cloud object storage (AWS S3 or Azure Blob). [Remove from this list if you implemented them.]
- Automated key rotation and external key management services.
- Rate limiting and advanced suspicious-activity detection.

12. Future Scope

- Deploy the platform to the cloud with HTTPS and managed databases.
- Containerize with Docker and add automated testing and CI/CD.
- Add multi-factor authentication and OAuth2 sign-in.
- Move file storage to S3 or Azure Blob Storage with envelope encryption and key rotation.
- Add rate limiting, anomaly detection and real-time security alerts.
- Add expiring links, download limits and file versioning.

- Perform formal security testing and performance benchmarking.

13. Conclusion

TrustShare was developed through four milestones, progressing from core authentication and file management to encryption, secure sharing, activity monitoring, analytics and testing. The core workflows of uploading, encrypting, sharing, revoking and monitoring files work and have been tested locally. The project demonstrates the main ideas of secure file sharing, and with deployment and further security verification it can be extended into a production-ready platform.

14. References

1. FastAPI documentation: https://fastapi.tiangolo.com
2. Next.js documentation: https://nextjs.org/docs
3. PostgreSQL documentation: https://www.postgresql.org/docs
4. Python cryptography library documentation: https://cryptography.io
5. NIST SP 800-38D, Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM)
6. OWASP Application Security Verification Standard: https://owasp.org